TripWise Agent

- Chatbot vs ReAct Agent
- Lab 3 - Group presentation

Problem

- Travel planning needs weather, attractions, budget and itinerary data.
- A baseline chatbot gives general suggestions.
- TripWise uses tools and observations to produce traceable plans.

Architecture

- User Request -> Thought -> Action: tool(args)
- -> Observation -> repeat -> Final Answer
- Runtime injects observations and records telemetry.

Tool Evolution

- Agent v1: weather, attractions, cost estimate, itinerary.
- Agent v2: route time, restaurants, budget check, weather risk.
- Goal: improve planning quality and budget awareness.

Telemetry Dashboard

- v1: 23 LLM calls | P50 15.3s | P99 56.0s | 13 tool calls
- v2: 11 LLM calls | P50 11.6s | P99 36.5s | 5 tool calls
- Caveat: v2 requests were stopped early by provider rate limits.

Failure Analysis

- 1. Endpoint returned 429 Too many requests.
- 2. Model sometimes returned XML-style <tool_call>.
- 3. Model sometimes wrote Observation before runtime injection.
- Implemented: XML parser fallback.
- Next: retry/backoff and stronger guardrails.

Live Demo

- python scripts/smoke_test.py
- python tripwise_agent.py --v2
- python scripts/parse_logs.py
- Demo query: "Goi y diem tham quan o Ha Noi"

Team & Conclusion

- 9 members: product, prompt, demo, tools, agent, evaluation and report.
- TripWise demonstrates ReAct execution and structured telemetry.
- The trace is the evidence for the next engineering iteration.